• service: UP • answers: UNKNOWN

SRE for LLMs

Observability and Chaos Engineering for AI

Ankush Sharma
Head of Infrastructure (SRE & AI Engineering), FloQast
Author, Observability for Large Language Models (Apress, 2026)

SREday San Francisco Q4
October 2, 2026 · 17:00 · Track 2



THE PROBLEM

The dashboard says healthy. The model is confidently wrong.

ALL SYSTEMS NOMINAL

HTTP 200	100%	p95	342 ms
errors	0.02%	GPU	61%

WHAT THE USER GOT

"Q3 revenue grew 14%, driven by the EMEA expansion."

No such expansion. No such number. Status 200, latency fine, alerts zero.

Hallucination

Fluent, confident, fabricated - the costliest 200 OK

Drift

Quality decays with no deploy event

Token burn

Retry loops + bloated context, bill climbs silently

Latency variance

TTFT spikes and long tails users feel

Silent degradation

Fallbacks + truncation: infra green, quality down

Five failure modes. Zero default alerts. up ≠ correct - detection has to move into the response path.

BLUEPRINT · STEP 1

SLOs for LLMs: two families, one contract

Family	SLI	Example target	Why this number
Performance	Time to First Token (TTFT)	`p90 < 500 ms`	Confirms the server is alive and processing
Performance	Inter-Token Latency (ITL)	`p95 < 50 ms`	Streaming feels smooth, like reading
Performance	Generation rate	`> 20 tok/s`	Humans read at 4-5 tok/s - stay ahead of the reader
Quality	Eval pass rate (groundedness)	`>= 97% on golden set`	Scored async by evaluators, not status codes
Quality	Hallucination rate (evaluated)	`< 1% weekly`	Burns the model-quality error budget
Quality	Refusal rate	`2-5% band`	Alert in both directions - too high and too low

Split the error budget: an infrastructure budget and a model-quality budget. A latency incident and a hallucination regression should both be able to freeze launches.

BLUEPRINT · STEP 2

Token metrics in, token cost down

EMIT - cost is a golden signal

prompt_tokens / completion_tokens

cost: request -> session -> feature

cache_hit_rate

context_window_utilization

prompt_latency vs completion_latency

tokens_per_finished_task

The last one is the unit that matters - a task, not a call. Context bloat predicts both cost and "lost in the middle" quality loss.

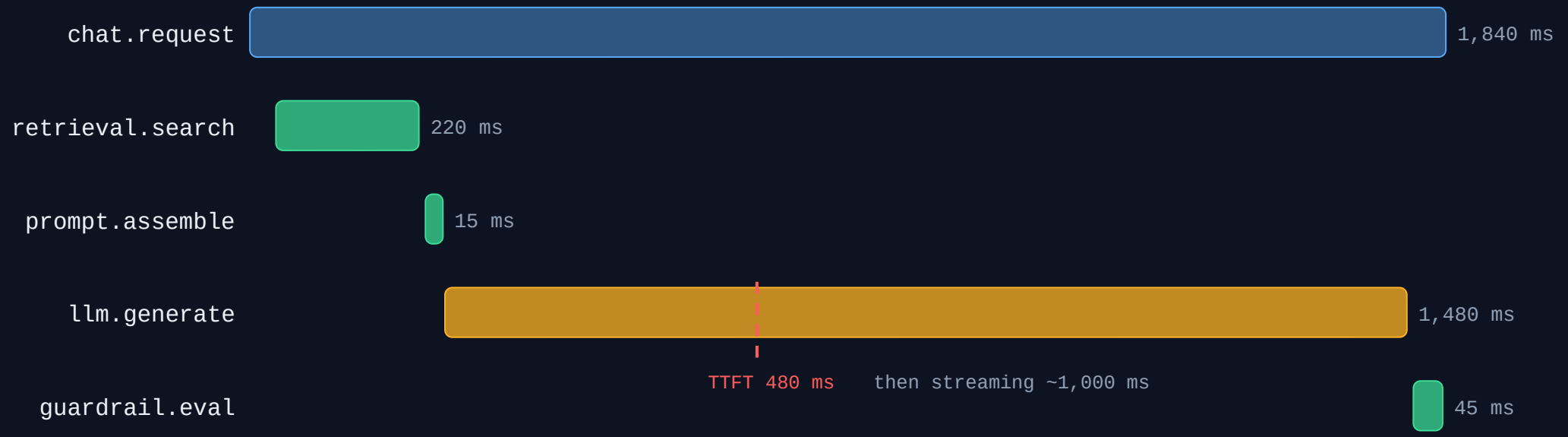
CUT - levers ranked by quality risk

Prompt caching - static first, dynamic last	**60-87% of input** *zero risk*
Model routing by task difficulty	**~58% of total bill** *gate w/ evals*
Thinking-token budgets per task class	**50-75% of reasoning** *low risk*
Compress predictable noise only	**70-90% of tool output** *never code/diffs*

~1,000× the tokens of plain prompting is what agentic workloads consume - and GitHub found that cutting tokens per CALL raised cost per TASK (shorter output -> recovery turns). Optimize the finished job.

BLUEPRINT · STEP 3

Instrument the pipeline: traces, logs, versions

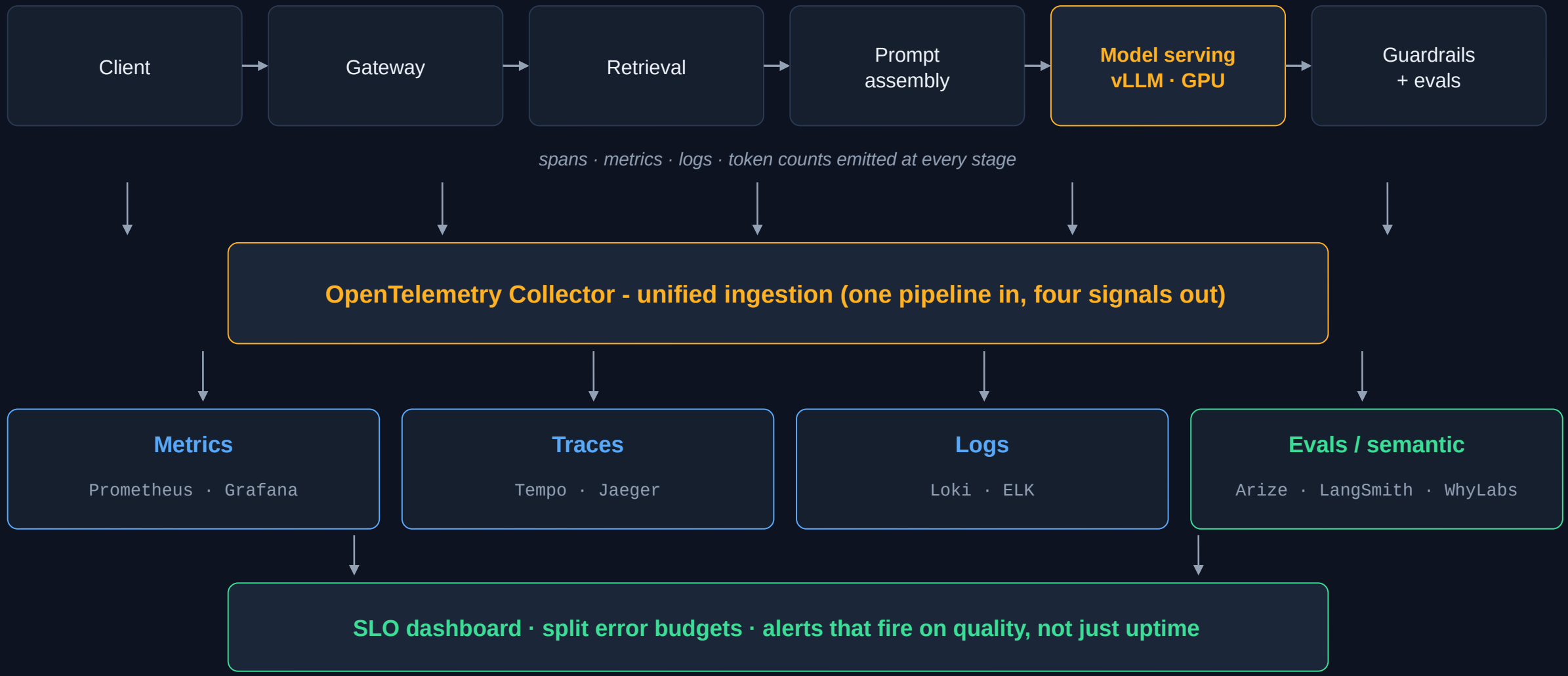


EVERY SPAN AND LOG LINE CARRIES

`model@version · prompt_template@rev · params · prompt/completion tokens · cache_hit · finish_reason · eval.score`

- Versions are non-negotiable - most "the model got worse" incidents are "someone edited the prompt template."
- PII redacted before persistence; full bodies sampled at 1-5%, plus 100% of failures.
- Use the OpenTelemetry GenAI semantic conventions - attribute names your vendors already understand.

One pipeline, one collector, one pane of glass



Break it on purpose - before Friday does

Why AI needs its own chaos discipline

- External model APIs fail in new ways - 429 storms, brownouts, silent model-version swaps you didn't schedule
- GPU serving is its own failure domain - VRAM pressure, request preemption, thermal throttling
- Nondeterminism hides partial failure - a degraded model still answers, just worse

STEADY STATE MUST BE DEFINED TWICE

```
infra:     http_5xx_rate < 1%  AND  latency_p99 < 5 s
semantic:  eval_pass_rate >= baseline on canary prompts
```

If you can't state "normal" as a query, you're not ready to inject failure.



The safety valve

Every experiment gets an automated abort switch:

```
IF 5xx_error_rate > 5%
OR latency_p99 > 5 s
FOR > 30 s
-> kill the chaos agent
```

Blast radius grows in steps: staging -> one canary cell -> a % of prod. Never all at once.

CHAOS FOR AI

The failure-injection catalog for model serving

Inject	Watch	Pass looks like
GPU memory pressure - VRAM-eater pod on the serving node	`dcgm_fi_dev_fb_used` `vllm:num_preemptions_total`	Latency rises, zero 5xx, requests queue gracefully
Provider outage / brownout - block egress to the model API	`fallback_route_count` `error_rate`	Router fails over in seconds; UX degrades, never dies
Latency injection on the inference path (toxiproxy / tc)	`ttft_p90, timeout_rate` `retry_count`	Timeouts bounded, retries jittered - no retry storm
429 rate-limit storm from the provider	`queue_depth, shed_rate`	Backpressure + load shedding; no cascade upstream
Degraded model swap - INT4-quantized fallback	`eval_pass_rate` `perplexity_delta`	Quality SLO catches it before any user ticket does
Context truncation / malformed output injection	`guardrail_block_rate` `json_parse_fail`	Parsers + guardrails contain it; blast radius = one request

Scope note: this catalog targets the model-serving layer. It composes with agent/tool-call chaos - Hazel Mahajan's 11:00 talk covered that layer beautifully.

CHAOS FOR AI

From game days to continuous chaos



1. Define

Steady state as code: infra queries + semantic baseline



2. Schedule

Run in CI/CD and off-peak windows - not just quarterly game days



3. Inject

One variable at a time, blast radius capped by config



4. Verify

Auto-compare SLIs vs. baseline; abort switch always armed



5. Report

Pass/fail report feeds the error-budget review

A MINIMAL RIG

`Locust / K6 for synthetic load · a chaos agent (fault injector) · Prometheus + Grafana to see the blast radius`

The hard part is step 1: a precise, mathematical definition of "normal" - bridging deterministic infrastructure and probabilistic model outputs.

CHAOS FOR AI

What the experiments actually taught us

Fallback routing is a muscle

Exercise failover weekly or it atrophies - the first real outage is the wrong time to discover a stale route config.

Degradation needs a quality floor

An INT4 fallback quadruples throughput but costs perplexity. Gate every degraded mode on evals, not vibes.

Cache is a resilience layer

Semantic caching isn't just a cost lever - cached answers keep the product alive through provider brownouts.

Watch preemptions, not utilization

vllm:num_preemptions_total moved before GPU utilization did. Leading indicators live one layer down.

THE BLUEPRINT

Eight steps to LLMs as boring as the rest of your stack



photograph this one

01 **Two SLO families**

performance (TTFT, ITL, tok/s) + quality (eval pass, hallucination, refusal)

02 **Split the error budget**

infrastructure vs. model quality - both can freeze launches

03 **Token + cost metrics**

per request, per session, per feature - cost is a golden signal

04 **Trace every hop**

retrieval -> assembly -> generate -> guardrail, on OTel GenAI conventions

05 **Version everything in logs**

prompt@rev, model@ver; redact PII; sample bodies + keep all failures

06 **One collector, one pane**

unified OTel ingestion - infra and semantic signals side by side

07 **Run the chaos catalog**

monthly, with the automated abort switch always armed

08 **Automate the loop**

steady state -> inject -> verify -> report, inside CI/CD

Make your LLMs as boring as the rest of your stack.

Thank you - questions welcome.



Get the book - Amazon

Observability for Large Language Models
(Apress, 2026)



Springer edition

link.springer.com/book/
10.1007/979-8-8688-2827-0



Let's connect

Ankush Sharma
linkedin.com/in/ankushsharmaa